

BÀI THỰC HÀNH DEEP LEARNING CHO KHOA HỌC DỮ LIỆU

LAB 2. Neural network fundamental

Hà Minh Tuấn

Khoa Toán - Tin học, ĐHKHTN, ĐHQG-HCM

Ngày 24 tháng 3 năm 2026

Hệ Thống Bài Tập Thực Hành Neural Networks Fundamentals với PyTorch

Họ và tên: _____ MSSV: _____

Hệ thống gồm hai loại bài tập:

- **Loại A** – Chỉ có yêu cầu, sinh viên tự viết code hoàn chỉnh từ đầu.
- **Loại B** – Có code mẫu nhưng *bị sai*. Sinh viên đọc yêu cầu, phân tích lỗi và sửa lại cho đúng. Lỗi có thể là: sai so với yêu cầu đề bài, hoặc sai cấu trúc PyTorch (dùng sai API, sai thứ tự, thiếu bước).

1 Giới Thiệu Mạng Nơ-ron và Perceptron

Bài A1 (Loại A) – Perceptron thủ công

Yêu cầu. Viết code Python/PyTorch thực hiện các bước sau:

1. Tạo một perceptron bằng `nn.Linear(3, 1)` với **3 đầu vào và 1 đầu ra**.
2. Gán thủ công trọng số $\mathbf{w} = [0.5, -0.3, 0.8]$ và bias $b = 0.2$ bằng `torch.no_grad()`.
3. Tạo tensor đầu vào $\mathbf{x} = [1.0, 2.0, -1.0]$.
4. Tính đầu ra trước kích hoạt (weighted sum + bias) và sau khi áp dụng hàm Sigmoid.
5. In ra: trọng số, bias, giá trị z (trước kích hoạt), và giá trị $\hat{y}$ (sau Sigmoid).

Kết quả mong đợi: $z = 0.5 \times 1.0 + (-0.3) \times 2.0 + 0.8 \times (-1.0) + 0.2 = -0.7$, $\hat{y} = \sigma(-0.7) \approx 0.3318$.

Bài B1 (Loại B) – Perceptron với Sigmoid

Yêu cầu. Tạo một perceptron `nn.Linear(2, 1)`, thực hiện forward pass với đầu vào $\mathbf{x} = [0.5, -1.0]$, sau đó áp dụng kích hoạt **Sigmoid** và in ra kết quả. Cuối cùng in thông báo xác nhận rằng perceptron đơn lớp chỉ giải được bài toán *phân tách tuyến tính*.

Code mẫu (có lỗi – hãy tìm và sửa):

```
import torch
import torch.nn as nn

torch.manual_seed(42)

# Tao perceptron 2 dau vao, 1 dau ra
perceptron = nn.Linear(2, 1)
sample_input = torch.tensor([0.5, -1.0])

# Forward pass
weighted_sum = perceptron(sample_input)

# Ap dung kích hoạt ReLU
output = torch.relu(weighted_sum)

print(f"Dau vao: {sample_input.numpy()}")
print(f"Trong so: {perceptron.weight.data.numpy()}")
print(f"Bias: {perceptron.bias.data.numpy()}")
print(f"z (truoc kích hoạt): {weighted_sum.item():.4f}")
print(f"Dau ra sau kích hoạt: {output.item():.4f}")
print("Luu y: Perceptron don lop chi giai quyet bai toan phi tuyến tính.")
```

Câu hỏi:

1. Liệt kê tất cả lỗi tìm được (sai yêu cầu hoặc sai cấu trúc).
2. Viết lại code đúng hoàn chỉnh.

2 Hàm Kích Hoạt

Bài A2 (Loại A) – So sánh các hàm kích hoạt

Yêu cầu. Viết code thực hiện:

1. Tạo vector đầu vào `x_vals = torch.linspace(-6, 6, 100)`.
2. Khởi tạo bốn hàm kích hoạt: `nn.Sigmoid()`, `nn.Tanh()`, `nn.ReLU()`, `nn.LeakyReLU(negative_slope=0.01)`.
3. Tính output của bốn hàm trên `x_vals`.
4. Vẽ bốn đồ thị trong một figure (2, 2) subplot, mỗi đồ thị có tiêu đề, nhãn trục và legend.
5. In ra khoảng giá trị `[min, max]` của từng hàm.

Bài A3 (Loại A) – Softmax và tổng xác suất

Yêu cầu. Viết code thực hiện:

1. Tạo tensor logit cho 2 mẫu, 3 lớp: `[[1.0, -0.5, 2.0], [0.1, 0.5, 0.2]]`.
2. Áp dụng `nn.Softmax(dim=1)` để tính xác suất.
3. Kiểm chứng rằng tổng xác suất của mỗi mẫu bằng 1.
4. In kết quả logit, output Softmax, và tổng xác suất từng mẫu.
5. Giải thích bằng comment tại sao cần `dim=1` chứ không phải `dim=0`.

Bài B2 (Loại B) – Vẽ đồ thị hàm kích hoạt

Yêu cầu. Tạo vector đầu vào từ -6 đến 6 gồm 100 điểm. Vẽ bốn hàm kích hoạt Sigmoid, Tanh, ReLU, Leaky ReLU trong bốn subplot (2×2). Mỗi subplot phải có: tiêu đề đúng tên hàm, nhãn trục x và y , grid mờ (`alpha=0.3`), và legend.

Code mẫu (có lỗi – hãy tìm và sửa):

```
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

x_vals = torch.linspace(-6, 6, 100)

sigmoid_fn = nn.Sigmoid()
tanh_fn = nn.Tanh()
relu_fn = nn.ReLU()
leaky_relu_fn = nn.LeakyReLU(negative_slope=0.01)

y_sigmoid = sigmoid_fn(x_vals)
y_tanh = tanh_fn(x_vals)
y_relu = relu_fn(x_vals)
y_leaky_relu = leaky_relu_fn(x_vals)

plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
```

```

plt.plot(x_vals.numpy(), y_sigmoid.numpy(),
         'b-', linewidth=2, label='Sigmoid')
plt.title('ReLU: max(0, x)')
plt.xlabel('x'); plt.ylabel('f(x)')
plt.grid(True, alpha=0.3); plt.legend()

plt.subplot(2, 2, 2)
plt.plot(x_vals.numpy(), y_tanh.numpy(),
         'r-', linewidth=2, label='Tanh')
plt.title('Tanh')
plt.xlabel('x'); plt.ylabel('f(x)')
plt.grid(True, alpha=0.3); plt.legend()

plt.subplot(2, 2, 3)
plt.plot(x_vals.numpy(), y_relu.numpy(),
         'g-', linewidth=2, label='ReLU')
plt.title('Sigmoid: 1 / (1 + exp(-x))')
plt.xlabel('x'); plt.ylabel('f(x)')
plt.grid(True, alpha=0.3); plt.legend()

plt.subplot(2, 2, 4)
plt.plot(x_vals.numpy(), y_leaky_relu.numpy(),
         'm-', linewidth=2, label='Leaky ReLU')
plt.title('Leaky ReLU (slope=0.1)')
plt.xlabel('x'); plt.ylabel('f(x)')
plt.grid(True, alpha=0.3); plt.legend()

plt.tight_layout()
plt.show()

```

Câu hỏi:

1. Liệt kê tất cả lỗi tìm được.
2. Viết lại code đúng hoàn chỉnh.

3 Mạng MLP và nn.Module

Bài A4 (Loại A) – Xây dựng MLP bằng nn.Module

Yêu cầu. Định nghĩa class `SimpleMLP` kế thừa `nn.Module` với kiến trúc:

$$\text{Input}(100) \xrightarrow{\text{fc1}} \text{Hidden}(64) \xrightarrow{\text{ReLU}} \xrightarrow{\text{fc2}} \text{Hidden}(32) \xrightarrow{\text{ReLU}} \xrightarrow{\text{fc3}} \text{Output}(5)$$

Sau đó:

1. Khởi tạo model và in kiến trúc bằng `print(model)`.
2. Đếm và in tổng số tham số có thể huấn luyện.
3. Tạo batch giả gồm 4 mẫu, mỗi mẫu có 100 đặc trưng bằng `torch.randn`.
4. Thực hiện forward pass bằng `torch.no_grad()` và in shape của output.

Bài A5 (Loại A) – Đếm tham số và kiểm tra shape

Yêu cầu. Dựa trên `SimpleMLP` đã xây dựng ở Bài A4:

1. Tính tay (trên giấy) số tham số của từng lớp Linear theo công thức: $n_{\text{params}} = n_{\text{in}} \times n_{\text{out}} + n_{\text{out}}$. Điền vào bảng:

Lớp	n_{in}	n_{out}	Số tham số
fc1	100	64	
fc2	64	32	
fc3	32	5	
Tổng			

2. Viết code kiểm chứng bằng vòng lặp `for p in model.parameters()`.
3. Kiểm tra shape đầu ra với các batch size khác nhau: $\{1, 4, 16, 32\}$ và giải thích tại sao shape luôn là `(batch_size, 5)`.

Bài B3 (Loại B) – Định nghĩa MLP

Yêu cầu. Định nghĩa class `SimpleMLP` kế thừa `nn.Module` với kiến trúc $\text{Input}(100) \rightarrow \text{Hidden}(64) \xrightarrow{\text{ReLU}} \rightarrow \text{Hidden}(32) \xrightarrow{\text{ReLU}} \rightarrow \text{Output}(5)$. Khởi tạo model, đếm tổng số tham số, tạo batch 4 mẫu và thực hiện forward pass, in shape của output.

Code mẫu (có lỗi – hãy tìm và sửa):

```

import torch
import torch.nn as nn

class SimpleMLP(nn.Module):
    def __init__(self, input_size, hidden_size, num_classes):
        # Thieu goi super().__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, hidden_size)
        self.relu2 = nn.ReLU()
        self.fc3 = nn.Linear(hidden_size // 2, num_classes)

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu1(out)
        out = self.fc2(out)
        out = self.relu2(out)
        out = self.fc3(out)
        return out

model = SimpleMLP(100, 64, 5)
print("Kien truc MLP:")
print(model)

total_params = sum(p.numel() for p in model.parameters()
                    if p.requires_grad)
print(f"Tong so tham so: {total_params:,}")

batch_size = 4
dummy_input = torch.randn(batch_size, 100)

predictions = model(dummy_input)
print(f"Shape dau ra: {predictions.shape}")

```

Câu hỏi:

1. Liệt kê tất cả lỗi tìm được (ghi rõ dòng và lý do).
2. Viết lại code đúng hoàn chỉnh.

4 Hàm Mất Mát

Bài A6 (Loại A) – Ba hàm mất mát cơ bản

Yêu cầu. Viết code minh hoạ ba hàm mất mát theo đúng trường hợp sử dụng:

1. **MSE Loss** (hồi quy): dự đoán $[1.0, 2.5, 3.8]$, nhãn thật $[1.2, 2.3, 4.0]$. Tính MSE và in kết quả.
2. **Cross-Entropy Loss** (phân loại đa lớp): logit $[[2.0, 0.5, -1.0], [0.1, 1.5, 0.2]]$, nhãn $[0, 1]$. Tính CE Loss và in kết quả.
3. **BCEWithLogitsLoss** (phân loại nhị phân): logit $[-0.5, 1.5, -2.0, 3.0]$, nhãn $[0.0, 1.0, 0.0, 1.0]$. Lưu ý reshape đúng shape trước khi tính.

Với mỗi hàm mất mát, in đầy đủ: tên hàm, input, nhãn, giá trị loss.

Bài B4 (Loại B) – MSE và Cross-Entropy Loss

Yêu cầu. Tính **MSE Loss** cho bài toán hồi quy với dự đoán $[1.0, 2.5, 3.8]$ và nhãn $[1.2, 2.3, 4.0]$. Tính **Cross-Entropy Loss** cho bài phân loại đa lớp với logit $[[2.0, 0.5, -1.0], [0.1, 1.5, 0.2]]$ và nhãn là *chỉ số lớp* $[0, 1]$. In kết quả rõ ràng cho từng hàm.

Code mẫu (có lỗi – hãy tìm và sửa):

```
import torch
import torch.nn as nn

# --- MSE Loss ---
loss_mse_fn = nn.MSELoss()
predictions_reg = torch.tensor([1.0, 2.5, 3.8])
targets_reg = torch.tensor([1.2, 2.3, 4.0])

mse = loss_mse_fn(predictions_reg, targets_reg)
print("MSE Loss (Regression):")
print(f" Predictions: {predictions_reg.numpy()}")
print(f" Targets: {targets_reg.numpy()}")
print(f" MSE Loss: {mse.item():.4f}")

# --- Cross-Entropy Loss ---
loss_ce_fn = nn.CrossEntropyLoss()
predictions_mc = torch.tensor([[2.0, 0.5, -1.0],
                                [0.1, 1.5, 0.2]])

# Nhan duoi dang one-hot thay vi chi so lop
targets_mc = torch.tensor([[1.0, 0.0, 0.0],
                            [0.0, 1.0, 0.0]])

ce = loss_ce_fn(predictions_mc, targets_mc)
print("\nCross-Entropy Loss (Multi-class):")
```

```
print(f"  Logits:\n  {predictions_mc.numpy()}")
print(f"  Targets: {targets_mc.numpy()}")
print(f"  CE Loss: {ce.item():.4f}")
```

Câu hỏi:

1. Liệt kê tất cả lỗi tìm được.
2. Viết lại code đúng hoàn chỉnh.

Bài B5 (Loại B) – BCEWithLogitsLoss

Yêu cầu. Tính BCEWithLogitsLoss cho bài toán phân loại nhị phân với logit $[-0.5, 1.5, -2.0, 3.0]$ (4 mẫu) và nhãn $[0.0, 1.0, 0.0, 1.0]$. Lưu ý: cần reshape tensor về shape (4, 1) trước khi đưa vào hàm loss.

Code mẫu (có lỗi – hãy tìm và sửa):

```
import torch
import torch.nn as nn

# BCEWithLogitsLoss -- phân loại nhị phân
loss_bce_fn = nn.BCELoss()          # Dung ham nay thay the
predictions_bc = torch.tensor([-0.5, 1.5, -2.0, 3.0])
targets_bc = torch.tensor([0.0, 1.0, 0.0, 1.0])

# Không reshape -- giữ nguyên shape (4,)
bce = loss_bce_fn(predictions_bc, targets_bc)

print("Binary Classification Loss:")
print(f"  Logits: {predictions_bc.numpy()}")
print(f"  Targets: {targets_bc.numpy()}")
print(f"  Loss: {bce.item():.4f}")
```

Câu hỏi:

1. Liệt kê tất cả lỗi tìm được.
2. Viết lại code đúng hoàn chỉnh.

5 Optimizer và Vòng Lặp Huấn Luyện

Bài A7 (Loại A) – Khởi tạo và thử nghiệm Optimizer

Yêu cầu. Viết code thực hiện:

1. Tạo model `nn.Linear(10, 2)`.
2. In giá trị ban đầu của `weight[0, 0]`.
3. Khởi tạo **hai optimizer**: SGD với `lr=0.01`, `momentum=0.9` và Adam với `lr=0.001`.
4. Tạo dữ liệu giả: input shape (5, 10), target shape (5, 2).
5. Thực hiện **một bước** tối ưu hoá với SGD theo đúng thứ tự: `zero_grad()` → `forward` → `loss` → `backward()` → `step()`.
6. In giá trị `weight[0, 0]` và `loss` sau bước cập nhật.

Bài B6 (Loại B) – Thứ tự bước tối ưu hoá

Yêu cầu. Tạo model `nn.Linear(10, 2)`, khởi tạo SGD optimizer với `lr=0.01`, `momentum=0.9`, tạo dữ liệu giả. Thực hiện **một bước** tối ưu hoá theo đúng thứ tự chuẩn và in giá trị tham số trước, sau cập nhật cùng với giá trị `loss`.

Code mẫu (có lỗi – hãy tìm và sửa):

```
import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(42)

dummy_model = nn.Linear(10, 2)
print(f"Gia tri ban dau weight[0,0]: "
      f"{dummy_model.weight[0,0].item():.4f}")

optimizer_sgd = optim.SGD(
    dummy_model.parameters(), lr=0.01, momentum=0.9)

dummy_input = torch.randn(5, 10)
dummy_target = torch.randn(5, 2)
criterion = nn.MSELoss()

# Thuc hien mot buoc toi uu hoa
outputs = dummy_model(dummy_input)      # Forward pass
loss = criterion(outputs, dummy_target)

loss.backward()                          # Tinh gradient
optimizer_sgd.zero_grad()                # Xoa gradient SAU backward
```

```
optimizer_sgd.step()                                # Cap nhat tham so

print(f"Gia tri sau buoc SGD: "
      f"{dummy_model.weight[0,0].item():.4f}")
print(f"Loss: {loss.item():.4f}")
```

Câu hỏi:

1. Liệt kê tất cả lỗi tìm được. Giải thích hậu quả của lỗi đó khi huấn luyện nhiều epoch.
2. Viết lại code đúng hoàn chỉnh.

6 Bài Toán XOR – Pipeline Đầy Đủ

Bài A8 (Loại A) – Chuẩn bị dữ liệu XOR

Yêu cầu. Viết code thực hiện đầy đủ bước chuẩn bị dữ liệu XOR:

1. Tạo tensor `X_xor` shape (4, 2) với bốn tổ hợp đầu vào XOR và tensor `y_xor` shape (4, 1) với nhãn tương ứng.
2. In bảng chân trị XOR đầy đủ.
3. Bọc dữ liệu bằng `TensorDataset` rồi tạo `DataLoader` với `batch_size=4` và `shuffle=True`.
4. In số lượng mẫu và batch size.
5. Lặp qua một batch từ `DataLoader` và in shape của input và label.

Bài A9 (Loại A) – Định nghĩa XORNet

Yêu cầu. Định nghĩa class `XORNet` kế thừa `nn.Module` với kiến trúc:

$$\text{Input}(2) \xrightarrow{\text{fc1, ReLU}} \text{Hidden}(8) \xrightarrow{\text{fc2}} \text{Output}(1)$$

Lớp output **không** có activation (sử dụng logit thô). Sau đó:

1. Khởi tạo và in kiến trúc.
2. Đếm và in tổng số tham số.
3. Thực hiện forward pass thử với input `[[0., 0.], [1., 1.]]`.

Bài A10 (Loại A) – Vòng lặp huấn luyện XOR

Yêu cầu. Dùng `XORNet` và dữ liệu XOR từ các bài trước, viết vòng lặp huấn luyện hoàn chỉnh:

1. Dùng `BCEWithLogitsLoss` và `Adam optimizer` với `lr=0.05`.
2. Huấn luyện **1000 epoch**, mỗi epoch duyệt qua `DataLoader`.
3. Trong mỗi batch: thực hiện đúng thứ tự `forward` → `loss` → `zero_grad()` → `backward()` → `step()`.
4. Lưu `loss` vào list và in `loss` mỗi 200 epoch.
5. Sau khi huấn luyện: vẽ đồ thị `loss` theo epoch với thang log (`yscale('log')`).

Bài A11 (Loại A) – Đánh giá XORNet

Yêu cầu. Sau khi huấn luyện `XORNet`, viết code đánh giá:

1. Chuyển sang chế độ `eval()` và bọc bằng `torch.no_grad()`.
2. Tính logit dự đoán cho toàn bộ `X_xor`.
3. Áp dụng Sigmoid để ra xác suất, rồi ngưỡng ≥ 0.5 để ra nhãn dự đoán.
4. Tính accuracy tổng thể.
5. In bảng kết quả chi tiết: Input | Nhãn thật | Xác suất | Nhãn dự đoán | Đúng/Sai.
6. Chuyển trở lại `train()` mode.

Bài B7 (Loại B) – Vòng lặp huấn luyện XOR

Yêu cầu. Huấn luyện `XORNet` giải bài toán XOR trong 1000 epoch. Dùng `BCEWithLogitsLoss`, `Adam lr=0.05`, `DataLoader batch_size=4`. Lưu `loss` mỗi epoch và in mỗi 200 epoch.

Code mẫu (có lỗi – hãy tìm và sửa):

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
```

```

torch.manual_seed(42)

X_xor = torch.tensor([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
y_xor = torch.tensor([[0.],[1.],[1.],[0.]])

xor_dataset = TensorDataset(X_xor, y_xor)
xor_dataloader = DataLoader(
    xor_dataset, batch_size=4, shuffle=True)

class XORNet(nn.Module):
    def __init__(self):
        super(XORNet, self).__init__()
        self.fc1 = nn.Linear(2, 8)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(8, 1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

xor_model = XORNet()
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(xor_model.parameters(), lr=0.05)

num_epochs = 1000
losses_history = []

for epoch in range(num_epochs):
    for inputs, labels in xor_dataloader:
        outputs = xor_model(inputs)
        loss = criterion(outputs, labels)

        # Sai thu tu: thieu zero_grad truoc backward
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

    losses_history.append(loss.item())

    if (epoch + 1) % 200 == 0:
        print(f"Epoch [{epoch+1}/{num_epochs}], "
              f"Loss: {loss.item():.4f}")

print(f"Hoan thanh! Loss cuoi: {loss.item():.4f}")

```

Câu hỏi:

1. Liệt kê tất cả lỗi tìm được. Giải thích tại sao thứ tự `zero_grad()` → `backward()` → `step()` là bắt buộc.

- Viết lại code đúng hoàn chỉnh.

Bài B8 (Loại B) – Đánh giá mô hình

Yêu cầu. Sau khi huấn luyện XORNet, chuyển sang chế độ evaluation, tính xác suất dự đoán cho toàn bộ tập XOR, tính accuracy và in bảng kết quả chi tiết. Chuyển trở lại training mode sau khi xong.

Code mẫu (có lỗi – hãy tìm và sửa):

```
# (Gia su xor_model da duoc huan luyen)

# Chuyển sang chế độ đánh giá
xor_model.train()                # Sai mode

# Tính dự đoán
test_logits = xor_model(X_xor)   # Thieu torch.no_grad()
test_probs  = torch.relu(test_logits) # Sai ham chuyen doi

predicted = (test_probs >= 0.5).float()
accuracy  = (predicted == y_xor).float().mean()

print(f"Accuracy: {accuracy.item()*100:.2f}%")
print("\nKet qua chi tiet:")
print("Input      | That | Prob  | Du doan | Dung?")
print("-" * 50)

for i in range(len(X_xor)):
    inp    = X_xor[i].numpy()
    true   = y_xor[i].item()
    prob   = test_probs[i].item()
    pred   = predicted[i].item()
    ok     = "Dung" if pred == true else "Sai"
    print(f"{inp} | {true:.0f} | {prob:.4f} | {pred:.0f} | {ok}")

# Không chuyển lại training mode
```

Câu hỏi:

- Liệt kê tất cả lỗi tìm được (ghi rõ từng lỗi và giải thích).

- Viết lại code đúng hoàn chỉnh.

7 Bài Tổng Hợp

Bài A12 (Loại A) – Pipeline đầy đủ cho bài toán NAND

Yêu cầu. Bài toán NAND: đầu ra là phủ định của AND. Bảng chân trị: $(0, 0) \rightarrow 1$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$.

Tự xây dựng pipeline hoàn chỉnh (không có code hướng dẫn):

- Tạo dữ liệu và DataLoader (batch_size = 4).
- Định nghĩa NANDNet: $2 \rightarrow 4 \xrightarrow{\text{ReLU}} 1$.
- Huấn luyện 500 epoch với BCEWithLogitsLoss và Adam lr=0.1.
- Vẽ loss curve theo epoch.
- Đánh giá và in bảng kết quả. Accuracy phải đạt 100%.

Bài B9 (Loại B) – Pipeline MLP phân loại đa lớp

Yêu cầu. Xây dựng và huấn luyện MLP phân loại 3 lớp trên dữ liệu tổng hợp gồm 60 mẫu với 4 đặc trưng. Kiến trúc: $4 \rightarrow 8 \xrightarrow{\text{ReLU}} 3$. Dùng CrossEntropyLoss và SGD lr=0.1. Huấn luyện 100 epoch (full-batch, không dùng DataLoader). In loss mỗi 20 epoch và tính accuracy cuối.

Code mẫu (có lỗi – hãy tìm và sửa):

```
import torch
import torch.nn as nn
import torch.optim as optim

torch.manual_seed(42)

# Dữ liệu tổng hợp: 60 mẫu, 4 đặc trưng, 3 lớp
X = torch.randn(60, 4)
y = torch.randint(0, 3, (60,))
```

```

class MLP3Class(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(4, 8)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(8, 3)

    def forward(self, x):
        return self.fc2(self.relu(self.fc1(x)))

model = MLP3Class()
# Dung MSELoss cho bai toan phan loai nhieu lop
criterion = nn.MSELoss()
optimizer = optim.SGD(model.parameters(), lr=0.1)

for epoch in range(100):
    outputs = model(X)

    # CrossEntropyLoss yeu cau nhan dang float
    y_float = y.float()
    loss = criterion(outputs, y_float)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1}: Loss = {loss.item():.4f}")

# Tinh accuracy
with torch.no_grad():
    preds = torch.sigmoid(model(X)) # Sai ham chuyen doi
    pred_cls = preds.argmax(dim=1)
    acc = (pred_cls == y).float().mean()
    print(f"Accuracy: {acc.item()*100:.2f}%")

```

Câu hỏi:

1. Liệt kê tất cả lỗi tìm được (ghi rõ từng lỗi và giải thích).
2. Viết lại code đúng hoàn chỉnh.